Deepfake Detection & Authenticity Verification System

Database SQL Files

Contains: Schema, Seed Data, Queries, Procedures, Functions, Triggers, Cursors, Transactions

01_schema.sql — Schema

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 01_schema.sql  |  DDL - Table Definitions & Constraints
--
================================================================================
====

-- Use a fresh database
CREATE DATABASE IF NOT EXISTS sentinel_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE sentinel_db;

-- Drop tables in reverse FK order (safe re-run)
DROP TABLE IF EXISTS EvidenceVault;
DROP TABLE IF EXISTS Notifications;
DROP TABLE IF EXISTS AuditLogs;
DROP TABLE IF EXISTS Scans;
DROP TABLE IF EXISTS VerdictTypes;
DROP TABLE IF EXISTS Users;
```

```
--
================================================================================
====
-- TABLE 1: VerdictTypes  (lookup / reference table — satisfies 3NF)
-- Extracted to eliminate transitive dependency:
--   Scans.RiskScore → VerdictName  (was a transitive dep; now FK
resolves it)
--
================================================================================
====
CREATE TABLE VerdictTypes (
    VerdictID    INT           NOT NULL AUTO_INCREMENT,
    VerdictName  VARCHAR(20)   NOT NULL,
    RiskMin      DECIMAL(5,2)  NOT NULL,
    RiskMax      DECIMAL(5,2)  NOT NULL,

    CONSTRAINT pk_VerdictTypes    PRIMARY KEY (VerdictID),
    CONSTRAINT uq_VerdictName     UNIQUE (VerdictName),
    CONSTRAINT chk_RiskRange      CHECK (RiskMin >= 0 AND RiskMax <= 100
AND RiskMin < RiskMax),
    CONSTRAINT chk_ValidVerdict   CHECK (VerdictName IN ('DEEPFAKE',
'SUSPICIOUS', 'AUTHENTIC'))
);

--
================================================================================
====
-- TABLE 2: Users
-- 1NF: All attributes atomic (Name kept as single field per synopsis
schema)
-- No partial or transitive dependencies (single-column PK)
--
================================================================================
====
CREATE TABLE Users (
    UserID        INT           NOT NULL AUTO_INCREMENT,
    Name          VARCHAR(100)  NOT NULL,
    Email         VARCHAR(255)  NOT NULL,
    Role          VARCHAR(20)   NOT NULL DEFAULT 'operative',
    PasswordHash  VARCHAR(255)  NOT NULL,
    OperativeID   VARCHAR(20)   NOT NULL,
    CreatedAt     DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_Users         PRIMARY KEY (UserID),
    CONSTRAINT uq_Email         UNIQUE (Email),
    CONSTRAINT uq_OperativeID   UNIQUE (OperativeID),
    CONSTRAINT chk_Role         CHECK (Role IN ('admin', 'operative',
'analyst'))
);

--
================================================================================
====
-- TABLE 3: Scans
-- VerdictID FK → VerdictTypes eliminates transitive dep
RiskScore→VerdictName
-- UNIQUE(FileHash) enforces duplicate prevention at DB level (trigger
backs this)
--
================================================================================
====
CREATE TABLE Scans (
    ScanID        INT            NOT NULL AUTO_INCREMENT,
    UserID        INT            NOT NULL,
    FileHash      VARCHAR(64)    NOT NULL,
    MediaType     VARCHAR(10)    NOT NULL,
    Status        VARCHAR(15)    NOT NULL DEFAULT 'PENDING',
    RiskScore     DECIMAL(5,2)   NULL,
    Confidence    DECIMAL(5,2)   NULL,
```

```
    VerdictID     INT            NULL,
    ModelVersion  VARCHAR(20)    NULL,
    CreatedAt     DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_Scans           PRIMARY KEY (ScanID),
    CONSTRAINT uq_FileHash        UNIQUE (FileHash),
    CONSTRAINT fk_Scans_User      FOREIGN KEY (UserID)    REFERENCES
Users(UserID)        ON DELETE CASCADE,
    CONSTRAINT fk_Scans_Verdict   FOREIGN KEY (VerdictID) REFERENCES
VerdictTypes(VerdictID) ON DELETE SET NULL,
    CONSTRAINT chk_MediaType      CHECK (MediaType IN ('VIDEO', 'AUDIO',
'IMAGE', 'UNKNOWN')),
    CONSTRAINT chk_Status         CHECK (Status IN ('PENDING',
'PROCESSING', 'COMPLETED', 'FAILED')),
    CONSTRAINT chk_RiskScore      CHECK (RiskScore IS NULL OR (RiskScore
>= 0 AND RiskScore <= 100)),
    CONSTRAINT chk_Confidence     CHECK (Confidence IS NULL OR
(Confidence >= 0 AND Confidence <= 100))
);

--
================================================================================
====
-- TABLE 4: AuditLogs
-- Records every significant user action for compliance and traceability
--
================================================================================
====
CREATE TABLE AuditLogs (
    AuditID    INT           NOT NULL AUTO_INCREMENT,
    UserID     INT           NOT NULL,
    Action     VARCHAR(100)  NOT NULL,
    Resource   VARCHAR(100)  NOT NULL,
    Timestamp  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_AuditLogs       PRIMARY KEY (AuditID),
    CONSTRAINT fk_AuditLogs_User  FOREIGN KEY (UserID) REFERENCES
Users(UserID) ON DELETE CASCADE
);

--
================================================================================
====
-- TABLE 5: Notifications
-- IsRead defaults to FALSE; Timestamp auto-set on insert
--
================================================================================
====
CREATE TABLE Notifications (
    NotificationID  INT           NOT NULL AUTO_INCREMENT,
    UserID          INT           NOT NULL,
    Type            VARCHAR(50)   NOT NULL,
    Message         TEXT          NOT NULL,
    IsRead          BOOLEAN       NOT NULL DEFAULT FALSE,
    Timestamp       DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_Notifications       PRIMARY KEY (NotificationID),
    CONSTRAINT fk_Notifications_User  FOREIGN KEY (UserID) REFERENCES
Users(UserID) ON DELETE CASCADE,
    CONSTRAINT chk_NotifType          CHECK (Type IN (
        'scan_complete', 'scan_failed', 'deepfake_detected',
        'user_mention', 'comment', 'share', 'system'
    ))
);

--
================================================================================
====
-- TABLE 6: EvidenceVault
```

```
-- Separate entity per synopsis (not embedded in Scans)
-- SHA256Hash unique — prevents duplicate evidence storage
--
================================================================================
====
CREATE TABLE EvidenceVault (
    EvidenceID   INT           NOT NULL AUTO_INCREMENT,
    ScanID       INT           NOT NULL,
    FilePath     VARCHAR(500)  NOT NULL,
    SHA256Hash   VARCHAR(64)   NOT NULL,
    CreatedAt    DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT pk_EvidenceVault        PRIMARY KEY (EvidenceID),
    CONSTRAINT uq_SHA256Hash           UNIQUE (SHA256Hash),
    CONSTRAINT fk_EvidenceVault_Scan   FOREIGN KEY (ScanID) REFERENCES
Scans(ScanID) ON DELETE CASCADE
);

--
================================================================================
====
-- INDEXES (performance for common query patterns)
--
================================================================================
====
CREATE INDEX idx_Scans_UserID    ON Scans(UserID);
CREATE INDEX idx_Scans_VerdictID ON Scans(VerdictID);
CREATE INDEX idx_Scans_Status    ON Scans(Status);
CREATE INDEX idx_Scans_CreatedAt ON Scans(CreatedAt);
CREATE INDEX idx_AuditLogs_UserID    ON AuditLogs(UserID);
CREATE INDEX idx_AuditLogs_Timestamp ON AuditLogs(Timestamp);
CREATE INDEX idx_Notifications_UserID ON Notifications(UserID);
CREATE INDEX idx_Notifications_IsRead ON Notifications(IsRead);
CREATE INDEX idx_EvidenceVault_ScanID ON EvidenceVault(ScanID);

--
================================================================================
====
-- NORMALIZATION PROOF SUMMARY
--
--------------------------------------------------------------------------------
----
-- 1NF: All columns hold atomic, single-valued data. No repeating groups.
--      (Name is one field; verdict info is in VerdictTypes, not repeated
strings)
--
-- 2NF: All non-key attributes fully depend on the single-column PK.
--      (No composite PKs exist, so 2NF is trivially satisfied)
--
-- 3NF: No transitive dependencies.
--      BEFORE: Scans had VerdictName, RiskMin, RiskMax → transitive on
RiskScore
--      AFTER:  VerdictTypes table holds these; Scans.VerdictID is just a
FK
--              Thus: ScanID → VerdictID → VerdictName (via FK, not
transitive dep)
--
-- BCNF: Every determinant is a candidate key.
--       VerdictTypes: VerdictID → all attributes (PK); VerdictName → all
(unique)
--       Users: UserID → all; Email → all (unique); OperativeID → all
(unique)
--       All tables satisfy BCNF.
--
================================================================================
====
```

02_seed.sql — Seed Data

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 02_seed.sql  |  DML - Sample Data Population
--
================================================================================
====

USE sentinel_db;

--
================================================================================
====
-- VerdictTypes — 3 rows, covers full 0–100 risk range
--
================================================================================
====
INSERT INTO VerdictTypes (VerdictName, RiskMin, RiskMax) VALUES
    ('AUTHENTIC',  0.00, 39.99),
    ('SUSPICIOUS', 40.00, 74.99),
    ('DEEPFAKE',   75.00, 100.00);

--
================================================================================
====
-- Users — 5 users (1 admin, 2 operatives, 2 analysts)
-- PasswordHash values are bcrypt hashes of 'Password@123'
--
================================================================================
====
INSERT INTO Users (Name, Email, Role, PasswordHash, OperativeID) VALUES
    ('Harshdeep Athawale', 'harshdeep@sentinel.io',  'admin',
'$2b$12$samplehash_admin_001',     'OPR-ADMIN-001'),
    ('Aakriti Chauhan',    'aakriti@sentinel.io',    'operative',
'$2b$12$samplehash_operative_002', 'OPR-FIELD-002'),
    ('Sehaj Dhillon',      'sehaj@sentinel.io',      'operative',
'$2b$12$samplehash_operative_003', 'OPR-FIELD-003'),
    ('Priya Sharma',       'priya@sentinel.io',      'analyst',
'$2b$12$samplehash_analyst_004',   'OPR-ANLYT-004'),
    ('Rahul Verma',        'rahul@sentinel.io',      'analyst',
'$2b$12$samplehash_analyst_005',   'OPR-ANLYT-005');

--
================================================================================
====
-- Scans — 12 scan records with varied statuses and verdicts
-- Note: FileHash values are SHA-256 hex strings (64 chars)
--
================================================================================
====
INSERT INTO Scans (UserID, FileHash, MediaType, Status, RiskScore,
Confidence, VerdictID, ModelVersion) VALUES
    -- Completed scans with verdicts
    (2,
'a3f1c2d4e5b6789012345678901234567890abcdef1234567890abcdef123456',
'VIDEO', 'COMPLETED', 87.50, 92.10, 3, 'v4'),
    (2,
'b4e2d3c5f6a7890123456789012345678901bcdef2345678901bcdef234567ab',
'IMAGE', 'COMPLETED', 23.00, 88.50, 1, 'v4'),
    (3,
'c5f3e4d6a7b8901234567890123456789012cdef3456789012cdef345678abcd',
'AUDIO', 'COMPLETED', 61.75, 79.30, 2, 'v4'),
    (3,
```

```
'd6a4f5e7b8c90123456789012345678901234567890123def4567890123def456789abcdef',
'VIDEO', 'COMPLETED', 91.20, 95.00, 3, 'v4'),
    (2,
'e7b5a6f8c9d01234567890123456789012345678901234ef5678901234ef567890abcdef01',
'IMAGE', 'COMPLETED', 15.40, 76.80, 1, 'v4'),
    (3,
'f8c6b7a9d0e12345678901234567890123456789012345f0678901235f067890abcdef0123',
'VIDEO', 'COMPLETED', 78.90, 89.60, 3, 'v4'),
    (2,
'09d7c8b0e1f23456789012345678901234560127890123601278901236012789 0abcdef0124',
'AUDIO', 'COMPLETED', 44.30, 82.10, 2, 'v4'),
    (3,
'10e8d9c1f2a34567890123456789012345671238901234712389012345671238901bcdef01245',
'IMAGE', 'COMPLETED', 5.20,  95.50, 1, 'v4'),
    -- In-progress scans
    (2,
'21f9e0d2a3b45678901234567890123456782349012345823490123456782349012cdef012456',
'VIDEO', 'PROCESSING', NULL, NULL, NULL, 'v4'),
    (3,
'32a0f1e3b4c56789012345678901234567893450123456934501234567893450123def0123456',
'IMAGE', 'PENDING',    NULL, NULL, NULL, 'v4'),
    -- Failed scan
    (4,
'43b1a2f4c5d67890123456789012345678904561234567045612345678904561234ef01234567',
'VIDEO', 'FAILED',     NULL, NULL, NULL, 'v4'),
    -- Analyst submitted scan
    (4,
'54c2b3a5d6e78901234567890123456789015672345678156723456789015672345f012345678',
'AUDIO', 'COMPLETED', 55.00, 70.00, 2, 'v4');

--
=============================================================================
====
-- AuditLogs — 16 audit entries covering login, upload, and verdict
events
--
=============================================================================
====
INSERT INTO AuditLogs (UserID, Action, Resource) VALUES
    (1, 'USER_LOGIN',       'auth'),
    (2, 'USER_LOGIN',       'auth'),
    (2, 'SCAN_SUBMITTED',   'scans'),
    (2, 'SCAN_SUBMITTED',   'scans'),
    (3, 'USER_LOGIN',       'auth'),
    (3, 'SCAN_SUBMITTED',   'scans'),
    (3, 'SCAN_SUBMITTED',   'scans'),
    (2, 'EVIDENCE_STORED',  'evidence_vault'),
    (3, 'EVIDENCE_STORED',  'evidence_vault'),
    (4, 'USER_LOGIN',       'auth'),
    (4, 'REPORT_GENERATED', 'reports'),
    (5, 'USER_LOGIN',       'auth'),
    (5, 'REPORT_GENERATED', 'reports'),
    (1, 'USER_CREATED',     'users'),
    (1, 'SYSTEM_CONFIG',    'system'),
    (2, 'SCAN_EXPORTED',    'scans');

--
=============================================================================
====
-- Notifications — 9 notifications for scan results and alerts
--
=============================================================================
====
INSERT INTO Notifications (UserID, Type, Message, IsRead) VALUES
    (2, 'scan_complete',    'Your scan (ScanID: 1) has been completed.
Verdict: DEEPFAKE.',         TRUE),
    (2, 'deepfake_detected','ALERT: Deepfake detected in your submission
with 87.5% risk score.',   TRUE),
    (2, 'scan_complete',    'Your scan (ScanID: 2) is complete. Verdict:
```

```
AUTHENTIC.',                FALSE),
    (3, 'scan_complete',    'Scan (ScanID: 3) complete. Verdict:
SUSPICIOUS — review recommended.', TRUE),
    (3, 'deepfake_detected','ALERT: Deepfake detected in scan ScanID: 4
with 91.2% risk score.',    FALSE),
    (3, 'scan_complete',    'Scan (ScanID: 6) complete. Verdict:
DEEPFAKE.',                         FALSE),
    (4, 'scan_failed',      'Scan (ScanID: 11) failed due to unsupported
codec. Please retry.',      FALSE),
    (5, 'system',           'System maintenance scheduled for 02:00 UTC.
Expect 15 min downtime.',   TRUE),
    (2, 'scan_complete',    'Scan (ScanID: 9) is now being
processed.',                            FALSE);

--
================================================================================
====
-- EvidenceVault — 6 evidence entries for completed DEEPFAKE/SUSPICIOUS
scans
--
================================================================================
====
INSERT INTO EvidenceVault (ScanID, FilePath, SHA256Hash) VALUES
    (1, '/vault/evidence/scan_001_deepfake_video.mp4',
'a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2'),
    (3, '/vault/evidence/scan_003_suspicious_audio.mp3',
'b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3'),
    (4, '/vault/evidence/scan_004_deepfake_video2.mp4',
'c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4'),
    (6, '/vault/evidence/scan_006_deepfake_video3.mp4',
'd4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5'),
    (7, '/vault/evidence/scan_007_suspicious_audio.mp3',
'e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6'),
    (12,'/vault/evidence/
scan_012_suspicious_audio2.mp3','f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1
b2c3d4e5f6a1b2c3d4e5f6a1');
```

03_queries.sql — Queries

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 03_queries.sql  |  DQL - SELECT Queries, JOINs, Aggregates,
Views
--
================================================================================
====

USE sentinel_db;

--
================================================================================
====
-- SECTION A: INNER JOINs
--
================================================================================
====

-- Q1: All completed scans with user name and verdict classification
SELECT
    s.ScanID,
    u.Name          AS SubmittedBy,
```

```
    u.OperativeID,
    s.MediaType,
    s.RiskScore,
    s.Confidence,
    vt.VerdictName  AS Verdict,
    s.ModelVersion,
    s.CreatedAt
FROM Scans s
INNER JOIN Users u          ON s.UserID    = u.UserID
INNER JOIN VerdictTypes vt  ON s.VerdictID = vt.VerdictID
WHERE s.Status = 'COMPLETED'
ORDER BY s.RiskScore DESC;

-- Q2: Evidence vault entries with scan details and submitting user
SELECT
    ev.EvidenceID,
    ev.FilePath,
    ev.SHA256Hash,
    s.ScanID,
    s.MediaType,
    s.RiskScore,
    vt.VerdictName AS Verdict,
    u.Name         AS SubmittedBy,
    ev.CreatedAt   AS StoredAt
FROM EvidenceVault ev
INNER JOIN Scans s         ON ev.ScanID   = s.ScanID
INNER JOIN Users u         ON s.UserID    = u.UserID
INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
ORDER BY ev.CreatedAt DESC;

-- Q3: Audit log with user name and role for all login events
SELECT
    al.AuditID,
    u.Name        AS UserName,
    u.Role,
    al.Action,
    al.Resource,
    al.Timestamp
FROM AuditLogs al
INNER JOIN Users u ON al.UserID = u.UserID
WHERE al.Action = 'USER_LOGIN'
ORDER BY al.Timestamp DESC;

--
================================================================================
====
-- SECTION B: LEFT JOINs
--
================================================================================
====

-- Q4: All users and their total scan count (include users with 0 scans)
SELECT
    u.UserID,
    u.Name,
    u.Role,
    u.OperativeID,
    COUNT(s.ScanID) AS TotalScans
FROM Users u
LEFT JOIN Scans s ON u.UserID = s.UserID
GROUP BY u.UserID, u.Name, u.Role, u.OperativeID
ORDER BY TotalScans DESC;

-- Q5: Users who have NEVER submitted a scan
SELECT
    u.UserID,
    u.Name,
    u.Email,
    u.Role
```

```
FROM Users u
LEFT JOIN Scans s ON u.UserID = s.UserID
WHERE s.ScanID IS NULL;

-- Q6: Scans with no evidence stored yet (potential oversight)
SELECT
    s.ScanID,
    u.Name    AS SubmittedBy,
    vt.VerdictName AS Verdict,
    s.RiskScore,
    s.CreatedAt
FROM Scans s
LEFT JOIN EvidenceVault ev  ON s.ScanID    = ev.ScanID
INNER JOIN Users u          ON s.UserID    = u.UserID
LEFT JOIN VerdictTypes vt   ON s.VerdictID = vt.VerdictID
WHERE s.Status = 'COMPLETED'
  AND ev.EvidenceID IS NULL;

--
================================================================================
====
-- SECTION C: GROUP BY + HAVING + Aggregates
--
================================================================================
====

-- Q7: Scan count per user grouped by role
SELECT
    u.Role,
    u.Name,
    COUNT(s.ScanID)       AS TotalScans,
    COUNT(CASE WHEN s.Status = 'COMPLETED' THEN 1 END) AS CompletedScans,
    COUNT(CASE WHEN s.Status = 'FAILED'    THEN 1 END) AS FailedScans
FROM Users u
LEFT JOIN Scans s ON u.UserID = s.UserID
GROUP BY u.UserID, u.Role, u.Name
ORDER BY TotalScans DESC;

-- Q8: Users with more than 2 completed scans (HAVING clause)
SELECT
    u.UserID,
    u.Name,
    u.OperativeID,
    COUNT(s.ScanID) AS CompletedScans
FROM Users u
INNER JOIN Scans s ON u.UserID = s.UserID
WHERE s.Status = 'COMPLETED'
GROUP BY u.UserID, u.Name, u.OperativeID
HAVING COUNT(s.ScanID) > 2
ORDER BY CompletedScans DESC;

-- Q9: Average, max, min risk score per verdict type
SELECT
    vt.VerdictName,
    COUNT(s.ScanID)         AS TotalScans,
    ROUND(AVG(s.RiskScore), 2) AS AvgRiskScore,
    MAX(s.RiskScore)           AS MaxRiskScore,
    MIN(s.RiskScore)           AS MinRiskScore,
    ROUND(AVG(s.Confidence), 2) AS AvgConfidence
FROM VerdictTypes vt
INNER JOIN Scans s ON vt.VerdictID = s.VerdictID
WHERE s.Status = 'COMPLETED'
GROUP BY vt.VerdictID, vt.VerdictName
ORDER BY vt.RiskMin DESC;

-- Q10: Verdict distribution as percentage (analytics dashboard)
SELECT
    vt.VerdictName,
    COUNT(s.ScanID)                                                    AS
```

```
Count,
    ROUND(COUNT(s.ScanID) * 100.0 / SUM(COUNT(s.ScanID)) OVER(), 1) AS
Percentage
FROM Scans s
INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
WHERE s.Status = 'COMPLETED'
GROUP BY vt.VerdictID, vt.VerdictName;

-- Q11: Users with at least one DEEPFAKE result (HAVING with JOIN)
SELECT
    u.Name,
    u.OperativeID,
    u.Role,
    COUNT(s.ScanID) AS DeepfakeCount
FROM Users u
INNER JOIN Scans s         ON u.UserID    = s.UserID
INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
WHERE vt.VerdictName = 'DEEPFAKE'
  AND s.Status = 'COMPLETED'
GROUP BY u.UserID, u.Name, u.OperativeID, u.Role
HAVING COUNT(s.ScanID) >= 1
ORDER BY DeepfakeCount DESC;

--
================================================================================
====
-- SECTION D: Subqueries
--
================================================================================
====

-- Q12: Scans with above-average risk score (correlated subquery)
SELECT
    s.ScanID,
    u.Name       AS SubmittedBy,
    s.RiskScore,
    vt.VerdictName AS Verdict
FROM Scans s
INNER JOIN Users u         ON s.UserID    = u.UserID
INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
WHERE s.RiskScore > (
    SELECT AVG(RiskScore)
    FROM Scans
    WHERE Status = 'COMPLETED' AND RiskScore IS NOT NULL
)
ORDER BY s.RiskScore DESC;

-- Q13: Users who have submitted at least one scan (WHERE EXISTS)
SELECT u.UserID, u.Name, u.Role
FROM Users u
WHERE EXISTS (
    SELECT 1
    FROM Scans s
    WHERE s.UserID = u.UserID
);

-- Q14: Scans submitted by operative-role users only (WHERE IN subquery)
SELECT
    s.ScanID,
    s.MediaType,
    s.Status,
    s.RiskScore,
    s.CreatedAt
FROM Scans s
WHERE s.UserID IN (
    SELECT UserID FROM Users WHERE Role = 'operative'
)
ORDER BY s.CreatedAt DESC;
```

```
-- Q15: Latest scan per user (correlated subquery for max)
SELECT
    u.Name,
    u.OperativeID,
    s.ScanID,
    s.Status,
    s.CreatedAt AS LastScanDate
FROM Users u
INNER JOIN Scans s ON s.ScanID = (
    SELECT ScanID
    FROM Scans s2
    WHERE s2.UserID = u.UserID
    ORDER BY s2.CreatedAt DESC
    LIMIT 1
);

--
================================================================================
====
-- SECTION E: Analytical Views (for dashboard)
--
================================================================================
====

-- View 1: Full scan summary joining all 3 main tables
CREATE OR REPLACE VIEW v_scan_summary AS
SELECT
    s.ScanID,
    u.Name          AS UserName,
    u.OperativeID,
    u.Role,
    s.MediaType,
    s.FileHash,
    s.Status,
    s.RiskScore,
    s.Confidence,
    vt.VerdictName  AS Verdict,
    s.ModelVersion,
    s.CreatedAt
FROM Scans s
INNER JOIN Users u          ON s.UserID    = u.UserID
LEFT  JOIN VerdictTypes vt  ON s.VerdictID = vt.VerdictID;

-- View 2: Per-user risk statistics for the analytics dashboard
CREATE OR REPLACE VIEW v_user_risk_stats AS
SELECT
    u.UserID,
    u.Name,
    u.OperativeID,
    u.Role,
    COUNT(s.ScanID)
AS TotalScans,
    COUNT(CASE WHEN s.Status = 'COMPLETED' THEN 1 END)
AS CompletedScans,
    COUNT(CASE WHEN vt.VerdictName = 'DEEPFAKE'   THEN 1 END)
AS DeepfakeCount,
    COUNT(CASE WHEN vt.VerdictName = 'SUSPICIOUS' THEN 1 END)
AS SuspiciousCount,
    COUNT(CASE WHEN vt.VerdictName = 'AUTHENTIC'  THEN 1 END)
AS AuthenticCount,
    ROUND(AVG(CASE WHEN s.Status = 'COMPLETED' THEN s.RiskScore END), 2)
AS AvgRiskScore,
    MAX(CASE WHEN s.Status = 'COMPLETED' THEN s.RiskScore END)
AS MaxRiskScore
FROM Users u
LEFT JOIN Scans s         ON u.UserID    = s.UserID
LEFT JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
GROUP BY u.UserID, u.Name, u.OperativeID, u.Role;
```

```
-- View 3: Complete audit trail with user context
CREATE OR REPLACE VIEW v_audit_trail AS
SELECT
    al.AuditID,
    u.Name        AS UserName,
    u.Role,
    u.OperativeID,
    al.Action,
    al.Resource,
    al.Timestamp
FROM AuditLogs al
INNER JOIN Users u ON al.UserID = u.UserID
ORDER BY al.Timestamp DESC;

-- View 4: Evidence vault with full context
CREATE OR REPLACE VIEW v_evidence_vault AS
SELECT
    ev.EvidenceID,
    u.Name          AS SubmittedBy,
    u.OperativeID,
    s.ScanID,
    s.MediaType,
    vt.VerdictName  AS Verdict,
    s.RiskScore,
    ev.FilePath,
    ev.SHA256Hash,
    ev.CreatedAt    AS StoredAt
FROM EvidenceVault ev
INNER JOIN Scans s         ON ev.ScanID   = s.ScanID
INNER JOIN Users u         ON s.UserID    = u.UserID
INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID;

-- View 5: Unread notification summary per user
CREATE OR REPLACE VIEW v_unread_notifications AS
SELECT
    u.Name,
    n.Type,
    n.Message,
    n.Timestamp
FROM Notifications n
INNER JOIN Users u ON n.UserID = u.UserID
WHERE n.IsRead = FALSE
ORDER BY n.Timestamp DESC;

-- Sample reads from views
SELECT * FROM v_scan_summary      ORDER BY CreatedAt DESC;
SELECT * FROM v_user_risk_stats   ORDER BY TotalScans DESC;
SELECT * FROM v_audit_trail       LIMIT 20;
SELECT * FROM v_evidence_vault;
SELECT * FROM v_unread_notifications;
```

04_procedures.sql — Procedures

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 04_procedures.sql  |  PL/SQL - Stored Procedures
--
================================================================================
====

USE sentinel_db;
```

```
DELIMITER $$

--
================================================================================
====
-- PROCEDURE 1: sp_submit_scan
-- Purpose : Validates user, checks hash uniqueness, inserts scan record,
--           auto-creates audit log entry — all within a single
transaction.
-- Params  : p_user_id      INT      — ID of submitting user
--           p_file_hash    VARCHAR  — SHA-256 hash of uploaded file
--           p_media_type   VARCHAR  — VIDEO / AUDIO / IMAGE / UNKNOWN
--           p_model_version VARCHAR — ML model version tag (e.g., 'v4')
--           OUT p_scan_id  INT      — Returns the new ScanID on success
--
================================================================================
====
DROP PROCEDURE IF EXISTS sp_submit_scan $$

CREATE PROCEDURE sp_submit_scan(
    IN  p_user_id      INT,
    IN  p_file_hash    VARCHAR(64),
    IN  p_media_type   VARCHAR(10),
    IN  p_model_version VARCHAR(20),
    OUT p_scan_id      INT
)
BEGIN
    DECLARE v_user_exists  INT DEFAULT 0;
    DECLARE v_hash_exists  INT DEFAULT 0;
    DECLARE v_new_scan_id  INT DEFAULT 0;

    -- Exception handler: rolls back and re-raises on any SQL error
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Step 1: Validate user exists
    SELECT COUNT(*) INTO v_user_exists
    FROM Users WHERE UserID = p_user_id;

    IF v_user_exists = 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'ERROR: User does not exist. Scan
submission aborted.';
    END IF;

    -- Step 2: Check for duplicate file hash
    SELECT COUNT(*) INTO v_hash_exists
    FROM Scans WHERE FileHash = p_file_hash;

    IF v_hash_exists > 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT = 'ERROR: Duplicate file hash detected. This
file has already been scanned.';
    END IF;

    -- Step 3: Insert scan record
    SAVEPOINT sp_scan_inserted;
    INSERT INTO Scans (UserID, FileHash, MediaType, Status, ModelVersion)
    VALUES (p_user_id, p_file_hash, p_media_type, 'PENDING',
p_model_version);
```

```
    SET v_new_scan_id = LAST_INSERT_ID();
    SET p_scan_id = v_new_scan_id;

    -- Step 4: Auto-create audit log entry
    SAVEPOINT sp_audit_inserted;
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (p_user_id, 'SCAN_SUBMITTED', CONCAT('scans/',
v_new_scan_id));

    COMMIT;

    SELECT CONCAT('Scan submitted successfully. ScanID: ', v_new_scan_id)
AS Result;
END $$


--
==========================================================================
====
-- PROCEDURE 2: sp_store_evidence
-- Purpose : Validates scan is COMPLETED, stores evidence in
EvidenceVault,
--           logs the action — with SAVEPOINT for partial rollback
safety.
-- Params  : p_scan_id    INT      — ScanID to attach evidence to
--           p_file_path  VARCHAR  — Storage path of evidence file
--           p_sha256     VARCHAR  — SHA-256 hash of evidence file
--
==========================================================================
====
DROP PROCEDURE IF EXISTS sp_store_evidence $$

CREATE PROCEDURE sp_store_evidence(
    IN p_scan_id   INT,
    IN p_file_path VARCHAR(500),
    IN p_sha256    VARCHAR(64)
)
BEGIN
    DECLARE v_scan_status  VARCHAR(15);
    DECLARE v_scan_userid  INT;
    DECLARE v_hash_exists  INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Step 1: Validate scan exists and is COMPLETED
    SELECT Status, UserID INTO v_scan_status, v_scan_userid
    FROM Scans WHERE ScanID = p_scan_id;

    IF v_scan_status IS NULL THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45002'
            SET MESSAGE_TEXT = 'ERROR: Scan not found.';
    END IF;

    IF v_scan_status != 'COMPLETED' THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45003'
            SET MESSAGE_TEXT = 'ERROR: Evidence can only be stored for
COMPLETED scans.';
    END IF;

    -- Step 2: Prevent duplicate evidence hash
    SELECT COUNT(*) INTO v_hash_exists
```

```
    FROM EvidenceVault WHERE SHA256Hash = p_sha256;

    IF v_hash_exists > 0 THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45004'
            SET MESSAGE_TEXT = 'ERROR: Duplicate SHA-256 hash. Evidence
already stored.';
    END IF;

    -- Step 3: Insert into EvidenceVault
    SAVEPOINT sp_evidence_inserted;
    INSERT INTO EvidenceVault (ScanID, FilePath, SHA256Hash)
    VALUES (p_scan_id, p_file_path, p_sha256);

    -- Step 4: Audit log entry
    SAVEPOINT sp_audit_inserted;
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (v_scan_userid, 'EVIDENCE_STORED', CONCAT('evidence_vault/
scan/', p_scan_id));

    COMMIT;

    SELECT CONCAT('Evidence stored successfully for ScanID: ', p_scan_id)
AS Result;
END $$


--
=============================================================================
====
-- PROCEDURE 3: sp_update_scan_verdict
-- Purpose : Resolves VerdictID from VerdictTypes based on risk score,
--           updates Scan record, notifies the scan owner.
-- Params  : p_scan_id      INT     — ScanID to update
--           p_risk_score   DECIMAL — Calculated risk score (0–100)
--           p_confidence   DECIMAL — Model confidence (0–100)
--           p_model_version VARCHAR — ML model version used
--
=============================================================================
====
DROP PROCEDURE IF EXISTS sp_update_scan_verdict $$

CREATE PROCEDURE sp_update_scan_verdict(
    IN p_scan_id       INT,
    IN p_risk_score    DECIMAL(5,2),
    IN p_confidence    DECIMAL(5,2),
    IN p_model_version VARCHAR(20)
)
BEGIN
    DECLARE v_verdict_id   INT;
    DECLARE v_verdict_name VARCHAR(20);
    DECLARE v_user_id      INT;
    DECLARE v_notif_msg    VARCHAR(300);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- Step 1: Resolve VerdictID from VerdictTypes by risk score range
    SELECT VerdictID, VerdictName INTO v_verdict_id, v_verdict_name
    FROM VerdictTypes
    WHERE p_risk_score BETWEEN RiskMin AND RiskMax
    LIMIT 1;

    IF v_verdict_id IS NULL THEN
```

```
        ROLLBACK;
        SIGNAL SQLSTATE '45005'
            SET MESSAGE_TEXT = 'ERROR: No matching verdict type for given
risk score.';
    END IF;

    -- Step 2: Get scan owner
    SELECT UserID INTO v_user_id
    FROM Scans WHERE ScanID = p_scan_id;

    -- Step 3: Update the Scan record
    SAVEPOINT sp_scan_updated;
    UPDATE Scans
    SET Status       = 'COMPLETED',
        RiskScore    = p_risk_score,
        Confidence   = p_confidence,
        VerdictID    = v_verdict_id,
        ModelVersion = p_model_version
    WHERE ScanID = p_scan_id;

    -- Step 4: Create notification for scan owner
    SAVEPOINT sp_notification_sent;
    SET v_notif_msg = CONCAT(
        'Scan #', p_scan_id, ' complete. Verdict: ', v_verdict_name,
        ' | Risk: ', p_risk_score, '% | Confidence: ', p_confidence, '%'
    );

    INSERT INTO Notifications (UserID, Type, Message)
    VALUES (
        v_user_id,
        CASE WHEN v_verdict_name = 'DEEPFAKE' THEN 'deepfake_detected'
ELSE 'scan_complete' END,
        v_notif_msg
    );

    -- Step 5: Audit log
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (v_user_id, 'VERDICT_ASSIGNED', CONCAT('scans/', p_scan_id));

    COMMIT;

    SELECT v_verdict_name AS Verdict, p_risk_score AS RiskScore,
p_confidence AS Confidence;
END $$


--
================================================================================
====
-- PROCEDURE 4: sp_generate_user_report
-- Purpose : Prints a scan summary report for a given user using a
cursor.
--           (Cursor logic is in 07_cursors.sql; this is the report entry
point)
--
================================================================================
====
DROP PROCEDURE IF EXISTS sp_generate_user_report $$

CREATE PROCEDURE sp_generate_user_report(
    IN p_user_id INT
)
BEGIN
    DECLARE v_user_name   VARCHAR(100);
    DECLARE v_total       INT DEFAULT 0;
    DECLARE v_deepfake    INT DEFAULT 0;
    DECLARE v_suspicious  INT DEFAULT 0;
    DECLARE v_authentic   INT DEFAULT 0;
    DECLARE v_avg_risk    DECIMAL(5,2);
```

```
    -- Validate user
    SELECT Name INTO v_user_name FROM Users WHERE UserID = p_user_id;

    IF v_user_name IS NULL THEN
        SIGNAL SQLSTATE '45006'
            SET MESSAGE_TEXT = 'ERROR: User not found for report
generation.';
    END IF;

    -- Aggregate statistics
    SELECT
        COUNT(*),
        COUNT(CASE WHEN vt.VerdictName = 'DEEPFAKE'   THEN 1 END),
        COUNT(CASE WHEN vt.VerdictName = 'SUSPICIOUS' THEN 1 END),
        COUNT(CASE WHEN vt.VerdictName = 'AUTHENTIC'  THEN 1 END),
        ROUND(AVG(s.RiskScore), 2)
    INTO v_total, v_deepfake, v_suspicious, v_authentic, v_avg_risk
    FROM Scans s
    LEFT JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
    WHERE s.UserID = p_user_id AND s.Status = 'COMPLETED';

    -- Return report
    SELECT
        v_user_name    AS UserName,
        v_total        AS TotalCompletedScans,
        v_deepfake     AS DeepfakeCount,
        v_suspicious   AS SuspiciousCount,
        v_authentic    AS AuthenticCount,
        v_avg_risk     AS AverageRiskScore;

    -- Audit the report generation
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (p_user_id, 'REPORT_GENERATED', CONCAT('reports/user/',
p_user_id));
END $$

DELIMITER ;

--
================================================================================
====
-- Test procedure calls
--
================================================================================
====
-- CALL sp_submit_scan(2,
'newhash_aabbccddeeff00112233445566778899aabbccddeeff0011223344',
'VIDEO', 'v4', @new_id);
-- SELECT @new_id;

-- CALL sp_update_scan_verdict(1, 87.50, 92.10, 'v4');

-- CALL sp_store_evidence(1, '/vault/evidence/scan_001.mp4',
'newevidence_sha256_112233445566778899aabbccddeeff00112233445566778899aa'
);

-- CALL sp_generate_user_report(2);
```

05_functions.sql — Functions

```
--
================================================================================
====
```

```
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 05_functions.sql  |  PL/SQL - User-Defined Functions
--
================================================================================

USE sentinel_db;

DELIMITER $$

--
================================================================================
-- FUNCTION 1: fn_validate_upload
-- Purpose : Returns TRUE if the file can be uploaded (no duplicate hash,
--           user exists and is valid). Returns FALSE with a reason
otherwise.
-- Params  : p_file_hash VARCHAR — SHA-256 hash of file to validate
--           p_user_id   INT     — ID of the user attempting the upload
-- Returns : TINYINT (1 = valid, 0 = invalid)
--
================================================================================
DROP FUNCTION IF EXISTS fn_validate_upload $$

CREATE FUNCTION fn_validate_upload(
    p_file_hash VARCHAR(64),
    p_user_id   INT
)
RETURNS TINYINT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_user_exists INT DEFAULT 0;
    DECLARE v_hash_exists INT DEFAULT 0;

    -- Check user exists
    SELECT COUNT(*) INTO v_user_exists
    FROM Users WHERE UserID = p_user_id;

    IF v_user_exists = 0 THEN
        RETURN 0;
    END IF;

    -- Check hash not already scanned
    SELECT COUNT(*) INTO v_hash_exists
    FROM Scans WHERE FileHash = p_file_hash;

    IF v_hash_exists > 0 THEN
        RETURN 0;
    END IF;

    RETURN 1;
END $$


--
================================================================================
-- FUNCTION 2: fn_get_verdict_label
-- Purpose : Given a numeric risk score, returns the verdict label string
--           by querying the VerdictTypes reference table.
-- Params  : p_risk_score DECIMAL — Risk score between 0.00 and 100.00
-- Returns : VARCHAR (DEEPFAKE / SUSPICIOUS / AUTHENTIC / UNKNOWN)
--
================================================================================
DROP FUNCTION IF EXISTS fn_get_verdict_label $$
```

```
CREATE FUNCTION fn_get_verdict_label(
    p_risk_score DECIMAL(5,2)
)
RETURNS VARCHAR(20)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_label VARCHAR(20) DEFAULT 'UNKNOWN';

    SELECT VerdictName INTO v_label
    FROM VerdictTypes
    WHERE p_risk_score BETWEEN RiskMin AND RiskMax
    LIMIT 1;

    RETURN v_label;
END $$


--
================================================================================
====
-- FUNCTION 3: fn_count_user_scans
-- Purpose : Returns the count of completed scans for a user filtered by
--           a given verdict label (or 'ALL' to count all completed
scans).
-- Params  : p_user_id INT      — User to query
--           p_verdict VARCHAR  — 'DEEPFAKE' | 'SUSPICIOUS' | 'AUTHENTIC'
| 'ALL'
-- Returns : INT count
--
================================================================================
====
DROP FUNCTION IF EXISTS fn_count_user_scans $$

CREATE FUNCTION fn_count_user_scans(
    p_user_id INT,
    p_verdict VARCHAR(20)
)
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count INT DEFAULT 0;

    IF p_verdict = 'ALL' THEN
        SELECT COUNT(*) INTO v_count
        FROM Scans
        WHERE UserID = p_user_id AND Status = 'COMPLETED';
    ELSE
        SELECT COUNT(*) INTO v_count
        FROM Scans s
        INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
        WHERE s.UserID = p_user_id
          AND s.Status = 'COMPLETED'
          AND vt.VerdictName = p_verdict;
    END IF;

    RETURN v_count;
END $$


--
================================================================================
====
-- FUNCTION 4: fn_user_threat_level
-- Purpose : Computes an aggregated threat level for a user based on
their
--           overall deepfake ratio. Returns 'HIGH' / 'MEDIUM' / 'LOW'.
```

```
--           Useful for admin risk dashboard.
-- Params  : p_user_id INT — User to evaluate
-- Returns : VARCHAR (HIGH / MEDIUM / LOW / INSUFFICIENT_DATA)
--
================================================================================
====
DROP FUNCTION IF EXISTS fn_user_threat_level $$

CREATE FUNCTION fn_user_threat_level(
    p_user_id INT
)
RETURNS VARCHAR(20)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total    INT DEFAULT 0;
    DECLARE v_deepfake INT DEFAULT 0;
    DECLARE v_ratio    DECIMAL(5,2) DEFAULT 0;

    SELECT COUNT(*) INTO v_total
    FROM Scans WHERE UserID = p_user_id AND Status = 'COMPLETED';

    IF v_total < 3 THEN
        RETURN 'INSUFFICIENT_DATA';
    END IF;

    SELECT COUNT(*) INTO v_deepfake
    FROM Scans s
    INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
    WHERE s.UserID = p_user_id
      AND s.Status = 'COMPLETED'
      AND vt.VerdictName = 'DEEPFAKE';

    SET v_ratio = (v_deepfake * 100.0) / v_total;

    IF v_ratio >= 60 THEN
        RETURN 'HIGH';
    ELSEIF v_ratio >= 30 THEN
        RETURN 'MEDIUM';
    ELSE
        RETURN 'LOW';
    END IF;
END $$

DELIMITER ;

--
================================================================================
====
-- Function usage examples
--
================================================================================
====
-- SELECT
fn_validate_upload('a3f1c2d4e5b6789012345678901234567890abcdef1234567890a
bcdef123456', 2) AS IsValid;
-- SELECT fn_get_verdict_label(87.5)  AS Verdict;  -- Returns DEEPFAKE
-- SELECT fn_get_verdict_label(55.0)  AS Verdict;  -- Returns SUSPICIOUS
-- SELECT fn_get_verdict_label(20.0)  AS Verdict;  -- Returns AUTHENTIC
-- SELECT fn_count_user_scans(2, 'DEEPFAKE')  AS DeepfakeCount;
-- SELECT fn_count_user_scans(2, 'ALL')        AS TotalScans;
-- SELECT fn_user_threat_level(2)              AS ThreatLevel;
-- SELECT fn_user_threat_level(3)              AS ThreatLevel;

-- Use functions inline in queries
SELECT
    u.Name,
    fn_count_user_scans(u.UserID, 'ALL')       AS TotalScans,
    fn_count_user_scans(u.UserID, 'DEEPFAKE')  AS Deepfakes,
```

```
    fn_user_threat_level(u.UserID)             AS ThreatLevel
FROM Users u
WHERE u.Role = 'operative';
```

06_triggers.sql — Triggers

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 06_triggers.sql  |  PL/SQL - Database Triggers
--
================================================================================
====

USE sentinel_db;

DELIMITER $$

--
================================================================================
====
-- TRIGGER 1: trg_prevent_duplicate_hash
-- Event   : BEFORE INSERT ON Scans
-- Purpose : Enforces duplicate file hash prevention at the database
level.
--           Even if application-layer check is bypassed, the DB blocks
it.
-- Result  : Raises SQLSTATE '45001' if FileHash already exists in Scans.
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_prevent_duplicate_hash $$

CREATE TRIGGER trg_prevent_duplicate_hash
BEFORE INSERT ON Scans
FOR EACH ROW
BEGIN
    DECLARE v_hash_count INT DEFAULT 0;

    SELECT COUNT(*) INTO v_hash_count
    FROM Scans
    WHERE FileHash = NEW.FileHash;

    IF v_hash_count > 0 THEN
        SIGNAL SQLSTATE '45001'
            SET MESSAGE_TEXT = 'TRIGGER ERROR: Duplicate file hash. This
media has already been submitted.';
    END IF;
END $$


--
================================================================================
====
-- TRIGGER 2: trg_auto_audit_on_scan_insert
-- Event   : AFTER INSERT ON Scans
-- Purpose : Automatically creates an AuditLog entry every time a scan is
--           submitted, ensuring no scan submission goes unlogged.
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_auto_audit_on_scan_insert $$
```

```
CREATE TRIGGER trg_auto_audit_on_scan_insert
AFTER INSERT ON Scans
FOR EACH ROW
BEGIN
    INSERT INTO AuditLogs (UserID, Action, Resource, Timestamp)
    VALUES (
        NEW.UserID,
        'SCAN_SUBMITTED',
        CONCAT('scans/', NEW.ScanID),
        NOW()
    );
END $$


--
================================================================================
====
-- TRIGGER 3: trg_auto_notify_on_verdict
-- Event   : AFTER UPDATE ON Scans
-- Purpose : Fires when a scan's Status changes to 'COMPLETED'.
--           If verdict is DEEPFAKE, sends a 'deepfake_detected' alert.
--           For all completions, sends a 'scan_complete' notification.
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_auto_notify_on_verdict $$

CREATE TRIGGER trg_auto_notify_on_verdict
AFTER UPDATE ON Scans
FOR EACH ROW
BEGIN
    DECLARE v_verdict_name VARCHAR(20) DEFAULT NULL;
    DECLARE v_message      VARCHAR(300);
    DECLARE v_notif_type   VARCHAR(30);

    -- Only fire when status transitions to COMPLETED
    IF OLD.Status != 'COMPLETED' AND NEW.Status = 'COMPLETED' THEN

        -- Resolve verdict label
        IF NEW.VerdictID IS NOT NULL THEN
            SELECT VerdictName INTO v_verdict_name
            FROM VerdictTypes
            WHERE VerdictID = NEW.VerdictID;
        END IF;

        -- Build notification type and message
        IF v_verdict_name = 'DEEPFAKE' THEN
            SET v_notif_type = 'deepfake_detected';
            SET v_message = CONCAT(
                'ALERT: Deepfake detected in scan #', NEW.ScanID,
                '. Risk Score: ', NEW.RiskScore, '% | Confidence: ',
NEW.Confidence, '%'
            );
        ELSE
            SET v_notif_type = 'scan_complete';
            SET v_message = CONCAT(
                'Scan #', NEW.ScanID, ' complete. Verdict: ',
IFNULL(v_verdict_name, 'PENDING'),
                '. Risk Score: ', IFNULL(NEW.RiskScore, 'N/A'), '%'
            );
        END IF;

        INSERT INTO Notifications (UserID, Type, Message, IsRead)
        VALUES (NEW.UserID, v_notif_type, v_message, FALSE);

    END IF;
END $$
```

```
--
================================================================================
====
-- TRIGGER 4: trg_set_verdict_on_risk_update
-- Event   : BEFORE UPDATE ON Scans
-- Purpose : When RiskScore is updated, automatically resolves and sets
--           VerdictID from VerdictTypes — no application code needed.
--           Ensures verdict is always consistent with the risk score
stored.
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_set_verdict_on_risk_update $$

CREATE TRIGGER trg_set_verdict_on_risk_update
BEFORE UPDATE ON Scans
FOR EACH ROW
BEGIN
    DECLARE v_resolved_verdict_id INT DEFAULT NULL;

    -- Only resolve if RiskScore is being set and is non-null
    IF NEW.RiskScore IS NOT NULL AND (OLD.RiskScore IS NULL OR
OLD.RiskScore != NEW.RiskScore) THEN
        SELECT VerdictID INTO v_resolved_verdict_id
        FROM VerdictTypes
        WHERE NEW.RiskScore BETWEEN RiskMin AND RiskMax
        LIMIT 1;

        SET NEW.VerdictID = v_resolved_verdict_id;
    END IF;
END $$


--
================================================================================
====
-- TRIGGER 5: trg_prevent_duplicate_evidence
-- Event   : BEFORE INSERT ON EvidenceVault
-- Purpose : Blocks duplicate SHA-256 evidence hashes at DB level.
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_prevent_duplicate_evidence $$

CREATE TRIGGER trg_prevent_duplicate_evidence
BEFORE INSERT ON EvidenceVault
FOR EACH ROW
BEGIN
    DECLARE v_hash_count INT DEFAULT 0;

    SELECT COUNT(*) INTO v_hash_count
    FROM EvidenceVault
    WHERE SHA256Hash = NEW.SHA256Hash;

    IF v_hash_count > 0 THEN
        SIGNAL SQLSTATE '45004'
            SET MESSAGE_TEXT = 'TRIGGER ERROR: Duplicate evidence hash.
This evidence has already been stored.';
    END IF;
END $$


--
================================================================================
====
-- TRIGGER 6: trg_audit_on_evidence_insert
-- Event   : AFTER INSERT ON EvidenceVault
-- Purpose : Logs every evidence storage action automatically.
```

```
--
================================================================================
====
DROP TRIGGER IF EXISTS trg_audit_on_evidence_insert $$

CREATE TRIGGER trg_audit_on_evidence_insert
AFTER INSERT ON EvidenceVault
FOR EACH ROW
BEGIN
    DECLARE v_scan_user_id INT;

    SELECT UserID INTO v_scan_user_id
    FROM Scans WHERE ScanID = NEW.ScanID;

    INSERT INTO AuditLogs (UserID, Action, Resource, Timestamp)
    VALUES (
        v_scan_user_id,
        'EVIDENCE_STORED',
        CONCAT('evidence_vault/', NEW.EvidenceID, '/scan/', NEW.ScanID),
        NOW()
    );
END $$

DELIMITER ;

--
================================================================================
====
-- Verify triggers are registered
--
================================================================================
====
SHOW TRIGGERS FROM sentinel_db;
```

07_cursors.sql — Cursors

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 07_cursors.sql  |  PL/SQL - Explicit Cursors for Report
Generation
--
================================================================================
====

USE sentinel_db;

DELIMITER $$

--
================================================================================
====
-- CURSOR BLOCK 1: cur_scan_summary
-- Purpose : Iterates all COMPLETED scans in a date range and prints a
--           tabular scan report (simulates analytics dashboard query).
-- Params  : p_start_date DATE, p_end_date DATE
--
================================================================================
====
DROP PROCEDURE IF EXISTS proc_scan_summary_report $$

CREATE PROCEDURE proc_scan_summary_report(
```

```
    IN p_start_date DATE,
    IN p_end_date   DATE
)
BEGIN
    -- Cursor variables
    DECLARE v_scan_id      INT;
    DECLARE v_user_name    VARCHAR(100);
    DECLARE v_operative_id VARCHAR(20);
    DECLARE v_media_type   VARCHAR(10);
    DECLARE v_verdict      VARCHAR(20);
    DECLARE v_risk_score   DECIMAL(5,2);
    DECLARE v_confidence   DECIMAL(5,2);
    DECLARE v_created_at   DATETIME;
    DECLARE v_done         BOOLEAN DEFAULT FALSE;

    -- Totals for summary footer
    DECLARE v_total_count     INT DEFAULT 0;
    DECLARE v_deepfake_count  INT DEFAULT 0;
    DECLARE v_suspicious_count INT DEFAULT 0;
    DECLARE v_authentic_count INT DEFAULT 0;

    -- Declare cursor
    DECLARE cur_scan_summary CURSOR FOR
        SELECT
            s.ScanID,
            u.Name,
            u.OperativeID,
            s.MediaType,
            vt.VerdictName,
            s.RiskScore,
            s.Confidence,
            s.CreatedAt
        FROM Scans s
        INNER JOIN Users u         ON s.UserID    = u.UserID
        INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
        WHERE s.Status = 'COMPLETED'
          AND DATE(s.CreatedAt) BETWEEN p_start_date AND p_end_date
        ORDER BY s.RiskScore DESC;

    -- NOT FOUND handler sets v_done = TRUE to exit loop
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    -- Create temp table to hold report rows
    DROP TEMPORARY TABLE IF EXISTS tmp_scan_report;
    CREATE TEMPORARY TABLE tmp_scan_report (
        ScanID      INT,
        UserName    VARCHAR(100),
        OperativeID VARCHAR(20),
        MediaType   VARCHAR(10),
        Verdict     VARCHAR(20),
        RiskScore   DECIMAL(5,2),
        Confidence  DECIMAL(5,2),
        ScannedAt   DATETIME
    );

    OPEN cur_scan_summary;

    read_loop: LOOP
        FETCH cur_scan_summary INTO
            v_scan_id, v_user_name, v_operative_id,
            v_media_type, v_verdict, v_risk_score,
            v_confidence, v_created_at;

        IF v_done THEN
            LEAVE read_loop;
        END IF;

        -- Insert into temp table
        INSERT INTO tmp_scan_report VALUES (
```

```
            v_scan_id, v_user_name, v_operative_id,
            v_media_type, v_verdict, v_risk_score,
            v_confidence, v_created_at
        );

        -- Accumulate counters
        SET v_total_count = v_total_count + 1;

        CASE v_verdict
            WHEN 'DEEPFAKE'   THEN SET v_deepfake_count   =
v_deepfake_count   + 1;
            WHEN 'SUSPICIOUS' THEN SET v_suspicious_count =
v_suspicious_count + 1;
            WHEN 'AUTHENTIC'  THEN SET v_authentic_count  =
v_authentic_count  + 1;
        END CASE;

    END LOOP;

    CLOSE cur_scan_summary;

    -- Output detailed rows
    SELECT * FROM tmp_scan_report ORDER BY RiskScore DESC;

    -- Output summary footer
    SELECT
        v_total_count      AS TotalScans,
        v_deepfake_count   AS Deepfakes,
        v_suspicious_count AS Suspicious,
        v_authentic_count  AS Authentic,
        p_start_date       AS PeriodStart,
        p_end_date         AS PeriodEnd;

    DROP TEMPORARY TABLE IF EXISTS tmp_scan_report;
END $$


--
================================================================================
====
-- CURSOR BLOCK 2: cur_deepfake_report
-- Purpose : Generates a threat intelligence report of all DEEPFAKE
verdicts
--           with user details — used by Analyst role for investigations.
--
================================================================================
====
DROP PROCEDURE IF EXISTS proc_deepfake_threat_report $$

CREATE PROCEDURE proc_deepfake_threat_report()
BEGIN
    DECLARE v_scan_id    INT;
    DECLARE v_user_name  VARCHAR(100);
    DECLARE v_op_id      VARCHAR(20);
    DECLARE v_role       VARCHAR(20);
    DECLARE v_media      VARCHAR(10);
    DECLARE v_risk       DECIMAL(5,2);
    DECLARE v_conf       DECIMAL(5,2);
    DECLARE v_model      VARCHAR(20);
    DECLARE v_ts         DATETIME;
    DECLARE v_done       BOOLEAN DEFAULT FALSE;
    DECLARE v_row_num    INT DEFAULT 0;

    DECLARE cur_deepfake_report CURSOR FOR
        SELECT
            s.ScanID,
            u.Name,
            u.OperativeID,
            u.Role,
```

```
            s.MediaType,
            s.RiskScore,
            s.Confidence,
            s.ModelVersion,
            s.CreatedAt
        FROM Scans s
        INNER JOIN Users u         ON s.UserID    = u.UserID
        INNER JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
        WHERE vt.VerdictName = 'DEEPFAKE'
          AND s.Status = 'COMPLETED'
        ORDER BY s.CreatedAt DESC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    DROP TEMPORARY TABLE IF EXISTS tmp_deepfake_report;
    CREATE TEMPORARY TABLE tmp_deepfake_report (
        RowNum      INT,
        ScanID      INT,
        UserName    VARCHAR(100),
        OperativeID VARCHAR(20),
        Role        VARCHAR(20),
        MediaType   VARCHAR(10),
        RiskScore   DECIMAL(5,2),
        Confidence  DECIMAL(5,2),
        ModelVersion VARCHAR(20),
        DetectedAt  DATETIME
    );

    OPEN cur_deepfake_report;

    df_loop: LOOP
        FETCH cur_deepfake_report INTO
            v_scan_id, v_user_name, v_op_id, v_role,
            v_media, v_risk, v_conf, v_model, v_ts;

        IF v_done THEN LEAVE df_loop; END IF;

        SET v_row_num = v_row_num + 1;

        INSERT INTO tmp_deepfake_report VALUES (
            v_row_num, v_scan_id, v_user_name, v_op_id,
            v_role, v_media, v_risk, v_conf, v_model, v_ts
        );
    END LOOP;

    CLOSE cur_deepfake_report;

    SELECT * FROM tmp_deepfake_report;
    SELECT v_row_num AS TotalDeepfakesDetected;

    DROP TEMPORARY TABLE IF EXISTS tmp_deepfake_report;
END $$


--
================================================================================
====
-- CURSOR BLOCK 3: cur_risk_analytics (Cursor FOR LOOP pattern)
-- Purpose : Per-user aggregate statistics: total scans, avg risk score,
--           deepfake count — simulates the analytics dashboard backend.
--
================================================================================
====
DROP PROCEDURE IF EXISTS proc_risk_analytics_report $$

CREATE PROCEDURE proc_risk_analytics_report()
BEGIN
    DECLARE v_user_id      INT;
    DECLARE v_name         VARCHAR(100);
```

```
    DECLARE v_op_id        VARCHAR(20);
    DECLARE v_total        INT;
    DECLARE v_deepfake     INT;
    DECLARE v_avg_risk     DECIMAL(5,2);
    DECLARE v_threat_level VARCHAR(20);
    DECLARE v_done         BOOLEAN DEFAULT FALSE;

    DECLARE cur_risk_analytics CURSOR FOR
        SELECT
            u.UserID,
            u.Name,
            u.OperativeID,
            COUNT(s.ScanID)                                          AS
TotalScans,
            COUNT(CASE WHEN vt.VerdictName = 'DEEPFAKE' THEN 1 END) AS
DeepfakeCount,
            ROUND(AVG(s.RiskScore), 2)                               AS
AvgRisk
        FROM Users u
        LEFT JOIN Scans s         ON u.UserID    = s.UserID AND s.Status
= 'COMPLETED'
        LEFT JOIN VerdictTypes vt ON s.VerdictID = vt.VerdictID
        GROUP BY u.UserID, u.Name, u.OperativeID;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    DROP TEMPORARY TABLE IF EXISTS tmp_risk_analytics;
    CREATE TEMPORARY TABLE tmp_risk_analytics (
        UserID      INT,
        UserName    VARCHAR(100),
        OperativeID VARCHAR(20),
        TotalScans  INT,
        Deepfakes   INT,
        AvgRiskScore DECIMAL(5,2),
        ThreatLevel VARCHAR(20)
    );

    OPEN cur_risk_analytics;

    ra_loop: LOOP
        FETCH cur_risk_analytics INTO
            v_user_id, v_name, v_op_id, v_total, v_deepfake, v_avg_risk;

        IF v_done THEN LEAVE ra_loop; END IF;

        -- Compute threat level inline
        IF v_total < 3 THEN
            SET v_threat_level = 'INSUFFICIENT_DATA';
        ELSEIF (v_deepfake * 100.0 / v_total) >= 60 THEN
            SET v_threat_level = 'HIGH';
        ELSEIF (v_deepfake * 100.0 / v_total) >= 30 THEN
            SET v_threat_level = 'MEDIUM';
        ELSE
            SET v_threat_level = 'LOW';
        END IF;

        INSERT INTO tmp_risk_analytics VALUES (
            v_user_id, v_name, v_op_id,
            v_total, v_deepfake, v_avg_risk, v_threat_level
        );
    END LOOP;

    CLOSE cur_risk_analytics;

    SELECT * FROM tmp_risk_analytics ORDER BY AvgRiskScore DESC;

    DROP TEMPORARY TABLE IF EXISTS tmp_risk_analytics;
END $$
```

```
--
================================================================================
====
-- CURSOR BLOCK 4: Parameterized cursor — user activity audit trail
-- Purpose : Retrieves all audit log entries for a specific user, ordered
--           by timestamp DESC — used by admin to review user activity.
--
================================================================================
====
DROP PROCEDURE IF EXISTS proc_user_activity_log $$

CREATE PROCEDURE proc_user_activity_log(
    IN p_user_id INT
)
BEGIN
    DECLARE v_audit_id INT;
    DECLARE v_action   VARCHAR(100);
    DECLARE v_resource VARCHAR(100);
    DECLARE v_ts       DATETIME;
    DECLARE v_done     BOOLEAN DEFAULT FALSE;
    DECLARE v_count    INT DEFAULT 0;

    -- Parameterized cursor using WHERE clause with IN parameter
    DECLARE cur_user_activity CURSOR FOR
        SELECT AuditID, Action, Resource, Timestamp
        FROM AuditLogs
        WHERE UserID = p_user_id
        ORDER BY Timestamp DESC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    DROP TEMPORARY TABLE IF EXISTS tmp_activity;
    CREATE TEMPORARY TABLE tmp_activity (
        AuditID   INT,
        Action    VARCHAR(100),
        Resource  VARCHAR(100),
        Timestamp DATETIME
    );

    OPEN cur_user_activity;

    act_loop: LOOP
        FETCH cur_user_activity INTO v_audit_id, v_action, v_resource,
v_ts;
        IF v_done THEN LEAVE act_loop; END IF;

        SET v_count = v_count + 1;
        INSERT INTO tmp_activity VALUES (v_audit_id, v_action,
v_resource, v_ts);
    END LOOP;

    CLOSE cur_user_activity;

    -- Output user name
    SELECT Name, Role, OperativeID FROM Users WHERE UserID = p_user_id;

    -- Output activity log
    SELECT * FROM tmp_activity;

    SELECT v_count AS TotalActions;

    DROP TEMPORARY TABLE IF EXISTS tmp_activity;
END $$

DELIMITER ;

--
================================================================================
```

```
====
-- Test cursor procedures
--
================================================================================
====
-- CALL proc_scan_summary_report('2024-01-01', '2026-12-31');
-- CALL proc_deepfake_threat_report();
-- CALL proc_risk_analytics_report();
-- CALL proc_user_activity_log(2);
```

08_transactions.sql — Transactions

```
--
================================================================================
====
-- SENTINEL: Deepfake Detection & Authenticity Verification System
-- UCS310 - Database Management System
-- File: 08_transactions.sql  |  Transaction Management — COMMIT/
ROLLBACK/SAVEPOINT
--
================================================================================
====

USE sentinel_db;

--
================================================================================
====
-- PRE-CLEANUP: Remove any leftover data from previous demo runs (safe
re-run)
--
================================================================================
====
DELETE FROM Scans WHERE FileHash IN (
    'txn1aabbccddeeff00112233445566778899aabbccddeeff0011223344556677',
    'concurrent112233aabbccddeeff00112233445566778899aabbccddeeff0011'
);
DELETE FROM EvidenceVault WHERE FilePath LIKE '%additional_frame%' OR
SHA256Hash LIKE 'txn3%';
DELETE FROM Users WHERE Email = 'test.operative@sentinel.io';

--
================================================================================
====
-- TRANSACTION SCENARIO 1: Full scan submission workflow
-- Demonstrates: START TRANSACTION, SAVEPOINT, COMMIT, ROLLBACK on error
-- Steps: insert scan → insert audit log → insert notification
--        If any step fails, rollback to appropriate savepoint or full
rollback
--
================================================================================
====

START TRANSACTION;

    -- SAVEPOINT before first write
    SAVEPOINT sp_start;

    -- Step 1: Insert scan record
    INSERT INTO Scans (UserID, FileHash, MediaType, Status, ModelVersion)
    VALUES (
        2,

'txn1aabbccddeeff00112233445566778899aabbccddeeff0011223344556677',
```

```
        'VIDEO',
        'PENDING',
        'v4'
    );

    SAVEPOINT sp_scan_inserted;

    -- Step 2: Insert audit log
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (2, 'SCAN_SUBMITTED', CONCAT('scans/', LAST_INSERT_ID()));

    SAVEPOINT sp_audit_inserted;

    -- Step 3: Insert notification
    INSERT INTO Notifications (UserID, Type, Message)
    VALUES (2, 'system', 'Your scan has been queued for processing.');

COMMIT;
-- All three inserts succeed atomically — either all commit or none do.


--
================================================================================
====
-- TRANSACTION SCENARIO 2: Verdict update with partial rollback using
SAVEPOINT
-- Demonstrates: ROLLBACK TO SAVEPOINT when notification step fails
(e.g.,
--               invalid NotifType) while keeping the scan update
committed.
--               Note: In real MySQL, only full ROLLBACK is possible
within
--               a single transaction; SAVEPOINT allows partial undo.
--
================================================================================
====

START TRANSACTION;

    SAVEPOINT sp_before_verdict_update;

    -- Step 1: Update scan with verdict result
    UPDATE Scans
    SET Status       = 'COMPLETED',
        RiskScore    = 82.50,
        Confidence   = 91.00,
        ModelVersion = 'v4'
    WHERE ScanID = 9;
    -- trg_set_verdict_on_risk_update trigger auto-sets VerdictID here

    SAVEPOINT sp_after_verdict_update;

    -- Step 2: Audit log
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (2, 'VERDICT_ASSIGNED', 'scans/9');

    SAVEPOINT sp_after_audit;

    -- Step 3: Notification (simulate partial failure — rollback only
this step)
    -- In a real scenario, if this INSERT fails due to a constraint
violation:
    -- ROLLBACK TO sp_after_audit;  -- undo only the notification, keep
the rest
    INSERT INTO Notifications (UserID, Type, Message)
    VALUES (2, 'deepfake_detected', 'ALERT: Scan #9 returned a DEEPFAKE
verdict. Risk: 82.5%');

COMMIT;
```

```
--
================================================================================
====
-- TRANSACTION SCENARIO 3: Evidence storage with duplicate hash detection
-- Demonstrates: ROLLBACK on constraint violation (duplicate SHA256Hash)
--
================================================================================
====

START TRANSACTION;

    SAVEPOINT sp_before_evidence;

    -- This WILL succeed (new unique hash)
    INSERT INTO EvidenceVault (ScanID, FilePath, SHA256Hash)
    VALUES (
        4,
        '/vault/evidence/scan_004_additional_frame.jpg',

'txn3ccddeeff00112233445566778899900aabbccddeeff00112233445566778 88'
    );

    SAVEPOINT sp_evidence_stored;

COMMIT;

-- Demonstrate ROLLBACK: attempt to insert duplicate SHA256Hash
-- Wrapped in a stored procedure so the trigger error is caught and
ROLLBACK executes gracefully
DROP PROCEDURE IF EXISTS demo_rollback_duplicate_evidence;
DELIMITER //
CREATE PROCEDURE demo_rollback_duplicate_evidence()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'ROLLBACK executed: Duplicate evidence hash blocked by
trigger. Transaction rolled back successfully.' AS TransactionResult;
    END;

    START TRANSACTION;
        SAVEPOINT sp_dup_attempt;
        -- trg_prevent_duplicate_evidence fires here and raises SQLSTATE
45004
        INSERT INTO EvidenceVault (ScanID, FilePath, SHA256Hash)
        VALUES (
            4,
            '/vault/evidence/scan_004_duplicate.jpg',

'a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2'  --
already exists (from seed)
        );
    COMMIT;
END //
DELIMITER ;
CALL demo_rollback_duplicate_evidence();


--
================================================================================
====
-- TRANSACTION SCENARIO 4: Concurrent scan submission isolation
-- Demonstrates: Setting isolation level + two users submitting same file
--
================================================================================
====
```

```
-- Set READ COMMITTED isolation level for this session
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

    -- User 2 submits a file
    SAVEPOINT sp_user2_scan;

    INSERT INTO Scans (UserID, FileHash, MediaType, Status, ModelVersion)
    VALUES (
        2,

'concurrent112233aabbccddeeff00112233445566778899aabbccddeeff0011',
        'IMAGE',
        'PENDING',
        'v4'
    );

    -- User 3 tries to submit the SAME file hash in another session
    -- (In a real concurrent scenario, session 2 would be blocked until
session 1 commits)
    -- The UNIQUE constraint + trigger ensures one will fail:
    -- INSERT INTO Scans (UserID, FileHash, MediaType, Status,
ModelVersion)
    -- VALUES (3, 'concurrent_hash_112233...', 'IMAGE', 'PENDING', 'v4');
    -- ^ This would raise: TRIGGER ERROR: Duplicate file hash.

COMMIT;


--
================================================================================
====
-- TRANSACTION SCENARIO 5: Atomic user creation + initial audit log
-- Demonstrates: Creating a user and logging the action atomically
--
================================================================================
====

START TRANSACTION;

    SAVEPOINT sp_new_user;

    INSERT INTO Users (Name, Email, Role, PasswordHash, OperativeID)
    VALUES (
        'Test Operative',
        'test.operative@sentinel.io',
        'operative',
        '$2b$12$samplehash_test_operative_txn5',
        'OPR-FIELD-006'
    );

    SET @new_user_id = LAST_INSERT_ID();
    SAVEPOINT sp_user_created;

    -- Admin (UserID=1) logs the creation action
    INSERT INTO AuditLogs (UserID, Action, Resource)
    VALUES (1, 'USER_CREATED', CONCAT('users/', @new_user_id));

    SAVEPOINT sp_audit_done;

    -- Send welcome notification to new user
    INSERT INTO Notifications (UserID, Type, Message)
    VALUES (
        @new_user_id,
        'system',
        'Welcome to SENTINEL. Your operative account is now active.'
    );
```

```
COMMIT;

-- Verify the transaction result
SELECT u.Name, u.Email, u.Role, u.OperativeID
FROM Users u
WHERE u.Email = 'test.operative@sentinel.io';


--
================================================================================
====
-- CLEANUP: Remove test data inserted by transaction demos
-- (Run this section after reviewing transaction results)
--
================================================================================
====
/*
DELETE FROM Scans WHERE FileHash LIKE 'txn%' OR FileHash LIKE
'concurrent%';
DELETE FROM Users WHERE Email = 'test.operative@sentinel.io';
DELETE FROM EvidenceVault WHERE FilePath LIKE '%additional_frame%' OR
FilePath LIKE '%txn3%';
*/

--
================================================================================
====
-- Reset isolation level to default
--
================================================================================
====
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```